Paradigmas de Programación
EIF400-II-2026- CLoria
Escuela de Informática – Universidad Nacional (UNA)
Proyecto Grupal: ***Proyecto Cyphail Sprint P2***
Versión 1 Octubre-2026 Preliminar

I. Contexto

Este `SPEC` es en el marco del proyecto `Cyphail` del curso `EIF400-II-2026-CLoria`. Se proponen los nuevos features a incorporar al proyecto en el sprint denominado `P2`. Se entiende en un contexto ágil y dinámico que puede requerir ajustes.

Expectativas Obligatorias:

- Se asume que el previo Sprint `P1` se ha alcanzado, es flexible y permite iterarse para conseguir lo acá planteado. Caso contrario se debe adaptar según corresponda.
- Se trabaja con el mismo modelo y lenguaje (Java) de proyecto, actualizando todo lo que corresponda (readme, tests, documentación, build, deploy a github y defensa).
- Todo lo que funcionaba antes debe seguir funcionando, excepto el denominado "fake engine" que simulaba el engine y queries.
- Al revisar el historial de commits se aprecia trabajo más allá de *tres días antes de la entrega* y participación de todo el equipo (push temprano y contínuo).
- Cualquier miembro del grupo defiende el código como su fuera propio y no usan la IA como *caja negra*.

Fechas y Modo de Entrega Definitivos:

- Se definirá la fecha definitiva oportunamente. Pero debe proyectarse P2 para subir el **Do 1/11**.
- Se sube al mismo drive y en la misma forma que P1 (`EIF400-II-2026-Cyphail-P2-NN-GG.zip` , (`NN-GG` el código de grupo de trabajo) que contiene el bundle `cyphail.bundle` del último commit.

Resumen Flujo Macro Esperable y Componentes

1. `Repl` en sesión de consola captura query

2. `Parser` construye el `AST` y la `tabla de variables` del estatuto.
3. `Codegen` transpila en los goals de Prolog (target) y la tabla de variables renombradas y aliases si los hay.
4. `Broker` recibe el target y hace la petición `MQI`. Inicia, mantiene una conexión con el engine vía un socket. La cierra al final de sesión. El broker levanta el proceso donde estará corriendo el engine (se acompaña este SPEC con un demo de ejemplo).
5. `Engine` ejecuta el query y responde vía `MQI` con el JSON.
6. `Broker` recibe y parsea (el demo de ejemplo lo hace usando `GSON`) el `JSON` de respuesta.
7. `Repl` despliega la respuesta.

II. Cambios y Nuevos Features

1. Repl: Permite estatutos multilínea para facilitar las pruebas. Terminan con ";".
2. Repl: Permite nuevo comando `.load` que lee de una ruta en disco un archivo `.cyphail`, lo parsea y ejecuta mostrando los resultados tabulares.

Ejemplo:

```
.load "somepath/mi_script.cyphail"
```

3. Parsing: Se parsean y visualizan (con `.tree`) los estatutos con **patrones de relaciones** y **propiedades de las mismas**. Ejemplo:

```
MATCH (p:Persona)-[:TRABAJA_EN]->(e:Empresa),
      (c:Persona)-[:TRABAJA_EN]->(e)
WHERE p.nombre <> c.nombre
RETURN p.nombre, c.nombre, e.nombre
```

4. Hay estatutos `CREATE` sin `MATCH` previo, `SET`, `REMOVE` y `DELETE` con patrones de relaciones. Ejemplos:

```
MATCH (a:Autor {nombre: 'Bill Gates'}),
      (b:Libro {titulo: 'Fake Windows Millenium'})
CREATE (a)-[:ESCRIBIO]->(b);
```

```
CREATE (a:Persona {nombre: 'Marta'})
    -[r:TRABAJA_EN {rol: 'Dev'}]->(e:Empresa
```

```
                                {nombre: 'UCR'})
RETURN a, r, e
```

```
MATCH (a:Person:Female)-[r:KNOWS]->(b:Person:Male)
WHERE r.since < 2000
DELETE r
```

```
MATCH (p:Product {code: "PROD-99"})
SET p.precio = 19.99
RETURN p
```

```
MATCH (p:Product {code: "PROD-99"})
REMOVE p.discount
RETURN p.id
```

```
MATCH
(u:Student {id: "4-666-000"})
-[r:attends]->
(c:Course {nrc: "50691"})
SET r.grade = 9.5
RETURN u.id, r.grade
```

5. Se crean un generador de código, o `Codgen` (un visitor del `AST`) que produce lo que se envía al engine de SWI-Prolog. El código generado son, en esencia, queries en formato de clásulas de Prolog. Esas clásulas las implementa el engine, las ejecuta y vía `MQI` devuelve un `JSON` de respuesta. Este debe procesarse para extraer los "bindings" para construir la salida tabular que el `repl` muestra.

6. Se validan condiciones de semántica estática (después de parsing) nuevas:

    - No repetir misma variable de relación en un mismo query: Por ejemplo, `(a-[r]->b), (c-[r]->b)` no se permite. **Nota**: Sí se puede repetir una de nodo (variable de join).

    - No se vale usar la misma variable de nodo como variable de relación. Ejemplo: No se permite `(a:N), (b)-[a]->c`.

7. Se restringe la sintaxis de una variable a que no pueda empezar con dos underscores (`__`). Por ejemplo, no se vale, `__person` como variable en `cyphail`. Esto es porque las variables del query deben ser convertidas a variables de Prolog. La idea es pasarlas a mayúscula y añadirles el prefijo `__` sin que choque con alguna variable ya definida en el fuente.

8. El generador `Codgen` genera a partir de los `ASTs` los `goals` de Prolog que el engine debe resolver. Esos goals son entendidos por el engine y se crean en Prolog mismo. Es decir, el `CodeGen` **transpila** `Cyphail` a código fuente (target) `Prolog`.
9. El fake engine `P1` para simular ejecución de queries desaparece y se implementa el engine que ejectuta los goals generados por la transpilación.

Modelo de `AST` y validación básica

Se adapta para manejar los nuevos casos. En el caso de relaciones la direccionalidad es un elemento a incorporar (`IN` , `OUT` , `NONE`).

II. Paradigma

1. FP Combinadores de Parser: se extienden los combinadores para los cambios planteados.
2. En la parte del `repl` usar programación funcional en código.

III Broker y Engine

1. Se adapta el broker para poder comunicarse con SWI-Prolog vía `MQI` , que responde en formato `JSON` siguiendo una estructura particular. Se adjuntará y explicará en clase un demo.
2. Se escribe el engine en SWI-PROLOG (se darán indicaciones en clase) siguiendo modelo de programación a explicar en clase.

`CodeGen` como Transpilador a Prolog

Para dar una idea del transpilador, se plantea un ejemplo. Asumimos que ya se crearon las relaciones usando `CREATE` (o manualmente para hacer pruebas). Asuma que el grafo se llama `amigos` .

- Query en `Cyphail` :

```
MATCH (a) -[:amigo]-> (b),
      (b) -[:amigo]-> (a)
WHERE a.age > 18 AND a.age = b.age
RETURN a.name, b.name
```

Posible Transpilación del Query a Prolog:

- Es un query y una secuencia de términos separados por comas (todo en un String). Podría verse así (notando que no tendremos un planificador y asumimos que la salida se genera en Prolog.

- El doble underscore lo usamos como marca de predicado creado por el sistema. Los predicados de prefijo `__cy` son del engine los `user` son del usuario que describen el grafo. La transpilación se vería así, probablemente. Asuma que el grafo se llama `amigos` y ya existe creado en Prolog. Los comentarios son solo para explicar el ejemplo, no se tienen que generar necesariamente.

- Target

```
cy_create_query( __QueryId,

    (query_01( __Aname, __Bname ) :-
     % (a) -[:amigo]-> (b)
     user_amigos_edge(__E1, amigo, __A, __B),
     % (b) -[:amigo]-> (a)
     user_amigos_edge(__E2, amigo, __B, __A),
     % Arco solo una vez
     cy_neq(__E1, __E2),
     % Obtenga a.age  y llámelo __Aage
     user_amigos_prop(__A, age, __Aage),
     % Filtre a.age > 18
     cy_gt(__ageA, 18),
     % Obtenga b.age  y llámelo __Bage
     cy_prop(B, age, __Bage),
     % Filtre a.age = b.age
     cy_eq(__Aage, __Bage),
     % Obtenga a.name  y llámelo __Aname
     user_amigos_prop(__A, name, __Aname),
     % Obtenga b.name  y llámelo __Bname
     user_amigos_prop(__B, name, __Bname)
    ),
),
% Ejecute el query con salida  en Rows.
% Recibe los nombres de la columnas
cy_exec_query(__QueryId, ['a.name', 'b.name'], Rows ),
% Borre el query
cy_erase_query(__Query_id )
```

- Las variables originales del query (`a` , y `b` en el ejemplo) deben ser renombradas a empezar en mayúscula pues eso es requerido por Prolog. (usamos `a` como `__A` , y `b` como `__B` en el ejemplo.) Note que hay nuevas variables de Prolog como

`__Aname` con cálculos a retornar (en este caso se refiere a `a.name`). Eso debe poderse manejar por parte del `Gencode`.

- En respuesta `MQI` devuelve un `JSON` como este, el que queda en función de `Rows`.
- Esta variable estará ligado a un string con el header y las filas, separadas por un cambio de línea, de la salida tabular:

```
{
  "args": [
   [
      [
       {"args": ["Rows", "a.name b.name\njuan  maria" ], "functor":"="
       }]
    ]
  ],
  "functor":"true"
}
```

El demo que acompaña a este SPEC ya hace la extracción desde el JSON usando `GSON`.

Restricción P2

- Si en el `RETURN` apareciera una variable de patrón, en ella se retornará solamente el `id` único del nodo o relación en el grafo. Esto para simplificar los alcances del prototipo. Lo normal en `Cypher` es que se retorne todo el objeto (sus labels y properties). Puede ser un extra.

Mapeo de Variables

- Nótese que el generador debe crear en Java una estructura de asociación de símbolos entre las variables originales del query, los cálculos y las del código generado en Prolog, para la correcta construcción de la tabular final en Java.